

Technology Stack

Date	29 September 2026
Team ID	SWTID-2026-7019
Project Name	EduGenie - AI Learning Assistant
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side	HTML5, CSS3 and vanilla JavaScript (Fetch API), rendered through Jinja2 templates	Single-page interface with one card per feature (Q&A, Explain, Summarize, Quiz, Learning Path). No build step or heavy framework is needed, so pages load fast, the code is easy to maintain, and the layout stays responsive on phones and laptops.
2	Backend / Server-Side	Python 3.13 with FastAPI, served by Uvicorn (ASGI); python-dotenv for configuration	FastAPI gives async REST endpoints (/qa, /explain/, /summarize/, /quiz, /learn/recommendations), request handling and JSON error responses with very little code. Python is also the natural language for the AI/ML libraries used by the project.
3	Database / Data Storage	No relational database (stateless design). Configuration in a .env file; Hugging Face model files cached on local disk	EduGenie generates answers on demand and does not need to store user accounts or history, so a database is unnecessary. The .env file keeps the API key and model names outside the source code; the local cache stores the LaMini-Flan-T5 weights after first download.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
4	Cloud / Hosting / Deployment	Local deployment with Uvicorn on port 8000 inside a Python virtual environment (venv); ready to containerise with Docker for cloud hosting	A virtual environment keeps dependencies isolated (requirements.txt) and the app runs with one command. Because the app is stateless it can be moved to a cloud platform such as Render, Azure or AWS behind HTTPS, and scaled horizontally when needed.
5	Version Control & CI/CD	Git and GitHub (repository with .gitignore); GitHub Actions planned for CI	GitHub supports team collaboration, code review and history. .gitignore keeps venv/, __pycache__/ and the .env secrets out of the repository. GitHub Actions can later run linting and automated tests on every push.
6	Third-Party APIs / Other Tools (AI generation)	Google Gemini API through the google-genai SDK (default model gemini-2.0-flash, configurable in .env)	Powers question answering, text summarization, quiz generation (JSON multiple-choice questions) and personalised learning-path recommendations, and is the fallback for topic explanations.
7	Third-Party APIs / Other Tools (local AI model)	Hugging Face Transformers, PyTorch, SentencePiece with the MBZUAI/LaMini-Fan-T5-783M model	Generates simple topic explanations for school students locally, without an API call. If the local model cannot be loaded, the app falls back to Gemini automatically.